计算复杂性函数的阶

- 核心思想：关注算法的运行时间是如何随着问题的规模（也即输入大小）而变化的

增长的阶

- 算法增长的阶也即**增长率**，**增长量级**或**时间复杂度**
- 只与输入问题的规模相关
- 一个算法比另一个算法“效率高”: 如果它的运算时间（原子操作步数）随着输入规模的增长比另一个算法增长的慢
- 只关注增长最快的项
- 常系数并不关键
- **典型增长阶：**
  - $O(1) < O(\log n) < O(\sqrt{n}) < O(n) < O(n\log n) < O\left(n^2\right) < O\left(n^3\right) < O\left(2^n\right) < O(n!)$
- **增长（渐近）记号：**
  - $O, \Theta, \Omega, o, \omega$

同阶函数集合

- 定义：

  $\Theta(g(n)) = \{f(n) \mid \exists c_1, c_2 > 0, \quad n_0, \forall n \geq n_0, 0 \leq c_1 g(n) \leq f(n) \leq c_2 g(n)\}$称为与$g(n)$同阶的函数集合。我们称为$g(n)$是$f(n)$的**渐进紧（确）界。**

  - 如果 $f(n) \in \Theta(g(n))$, $g(n)$ 与 $f(n)$ 同阶
  - $f(n) \in \Theta(g(n))$, 记作$f(n) = \Theta(g(n))$
  - 引入 c_1, c_2消除了常系数的影响
- 函数的例子：
  - 通常 $f(n) = an^2 + bn + c = \Theta\left(n^2\right)$, 其中 a, b, c 是常数且$a > 0$
  - $p(n) = \sum_{i=0}^{d} a_i n^i$, 其中 a_i是常数且 $a_d > 0, p(n) = \Theta\left(n^d\right)$
  - $\Theta\left(n^0\right)$或者$\Theta(1)$, 常数时间复杂性.

低阶函数集合

- 定义：

  对于给定的函数$g(n), O(g(n)) = \{f(n)$:存在正常数c和n_0满足对于所有$n \geq n_0, 0 \leq f(n) \leq cg(n)\}$，我们称为$g(n)$ 是 $f(n)$ 的**渐进上界。**

  - 记作$f(n) \in O(g(n))$，或者简记为$f(n) = O(g(n))$

高阶函数集合

- 定义：

  对于给定函数$g(n), \Omega(g(n)) = \{f(n)$存在正常数$c$和$n_0$, 使得对于所有$n \geq n_0, 0 \leq cg(n) \leq f(n)\}$

  - 记作：$f(n) \in \Omega(g(n))$，或简记为$f(n) = \Omega(g(n))$
  - 我们称为$g(n)$ 是$f(n)$ 的**渐进下界**
- 关于Ω标记
  - 可以用来描述运行时间的最好情况
  - 对所有输入都正确
  - 比如，对于插入排序：
    - 最好运行时间是 $\Omega(n)$
    - 最坏运行时间是$\Omega\left(n^2\right)$
    - 但是说运行时间是$\Omega\left(n^2\right)$则有误，因为直接说运行时间，不加修饰语，意味着需要对所有可能的输入都成立。
  - 可以用来描述问题
    - 排序问题的时间复杂性是$\Omega(n)$

三者关系

- $\Theta(g(n))$和$O(g(n))$的关系
  - $f(n) = \Theta(g(n)) \Rightarrow f(n) = O(g(n))$
  - $\Theta(g(n)) \subseteq O(g(n))$
  - Θ 标记强于 O 标记
  - **多项式界限**：如果$f(n) = O\left(n^k\right)$，k是常数，则称$f(n)$是多项式界限的。
- $\Theta(g(n))$、$O(g(n))$和$\Omega(g(n))$的关系
  - 对于$f(n)$和$g(n)$, $f(n) = \Theta(g(n))$ 当且仅当 $f(n) = O(g(n))$ 且 $f(n) = \Omega(g(n))$

严格低阶函数

- 定义：

  对于给定的函数$g(n), o(g(n)) = \{f(n)$:对于任意的正常数c，存在一个正数n_0，从而对所有$n \geq n_0, 0 \leq f(n) \leq cg(n)\}$

  - 记作$f(n) \in o(g(n))$，或者简记为$f(n) = o(g(n))$
- 关于o标记
  - O标记可能是或不是紧的

    $\therefore 2n^2 = O\left(n^2\right)$ 是紧的, 但 $2n = o\left(n^2\right)$不是紧的
  - o标记用于标记上界但**不是紧**的情况

    $\therefore 2n = o\left(n^2\right)$, 但是$2n^2 \neq o\left(n^2\right)$.

严格高阶函数

- 定义：

  $\omega(g(n)) = \{f(n)$: 对于任意正常数 c, 存在正数n_0对于 $n \geq n_0, \quad 0 \leq cg(n) < f(n)\}$

  - 记作：$f(n) \in \omega(g(n))$，或简记为$f(n) = \omega(g(n))$.
- 关于ω标记
  - ω表示**不紧的下界**.
  - $f(n) = \omega(g(n))$ 当且仅当 $g(n) = o(f(n))$.

等式与不等式中的渐近符号

- 当渐进符号单独出现在等式或不等式中时，例如 $f(n) = O(n^2)$, 此时意指集合的成员关系 $f(n) \in O(n^2)$。
- 当渐进符号出现在某个公式中时，例如$2n^2+3n+1=2n^2+O(n)$，此时意指$2n^2+3n+1=2n^2+f(n)$,其中f(n)是集合O(n)的某个函数，比如f(n) = 3n+1

渐近符号的性质

- 传递性: 所有五个标记
  - $f(n) = \Theta\big(g(n)\big)$ 且$g(n) = \Theta\big(h(n)\big)$ ➔ $f(n) = \Theta\big(h(n)\big)$
  - $f(n) = O\big(g(n)\big)$ 且$g(n) = O\big(h(n)\big)$ ➔ $f(n) = O\big(h(n)\big)$
  - $f(n) = \Omega\big(g(n)\big)$ 且$g(n) = \Omega\big(h(n)\big)$ ➔ $f(n) = \Omega\big(h(n)\big)$
  - $f(n) = o\big(g(n)\big)$ 且$g(n) = o\big(h(n)\big)$ ➔ $f(n) = o\big(h(n)\big)$
  - $f(n) = \omega\big(g(n)\big)$ 且$g(n) = \omega\big(h(n)\big)$ ➔ $f(n) = \omega\big(h(n)\big)$
- 自反性: Θ, O, Ω
  - $f(n) = \Theta\big(f(n)\big)$
  - $f(n) = O\big(f(n)\big)$
  - $f(n) = \Omega\big(f(n)\big)$

  为什么没有o, ω
- 对称性: Θ
  - $f(n) = \Theta\big(g(n)\big)$当且仅当$g\ (n) = \Theta\big(f(n)\big)$ 为什么?
- 反对称性:
  - $f(n) = O\big(g(n)\big)$当且仅当$g(n) = \Omega\big(f(n)\big)$
  - $f(n) = o\big(g(n)\big)$ 当且仅当$g(n) = \omega\big(f(n)\big)$

  为什么?

注意

并非所有函数都是渐进可比的，即对于函数*f(n)*和*g(n)*，可能*f(n)*≠ *O(g(n))* 且 *f(n)*≠ *Ω(g(n))*，例如 n和 $n^{1+sin(n)}$

$0 \leq 1+sin(n) \leq 2$

和式的估计

当一个算法包含循环体时，运行时间需要求和。

线性和

级数

1. 等差级数 $\sum_{k=0}^{n} k = \frac{n(n+1)}{2} = \Theta\left(n^2\right)$
2. 几何级数 $\sum_{k=0}^{n} x^k = 1 + x + x^2 + \cdots + x^n = \frac{x^{n+1}-1}{x-1} \quad (x \neq 1)$

   $\sum_{k=0}^{\alpha} x^k = \frac{1}{1-x} \quad (|x| < 1)$
3. 调和级数 $\sum_{k=0}^{n} \frac{1}{k} = 1 + \frac{1}{2} + \frac{1}{3} + \cdots + \frac{1}{n} = \ln n + O(1)$

$\sum_{k=1}^{n}(a_k - a_{k-1}) = a_n - a_0$.

$\sum_{k=0}^{n-1}(a_k - a_{k+1}) = a_0 - a_n$

4. 裂项级数 $\sum_{k=1}^{n-1}\frac{1}{k(k+1)} = \sum_{k=1}^{n-1}\left(\frac{1}{k} - \frac{1}{k+1}\right) = 1 - \frac{1}{n}$

$\lg\left(\prod_{k=1}^{n} a_k\right) = \sum_{k=1}^{n} \lg a_k$

计算和的界限

1. 采用**数学归纳法**（使用数学归纳法时，对于不同的n，使用相同的c值）
   - 例 1. 证明 $\sum_{k=0}^{n} 3^k = \mathrm{O}\left(3^n\right)$

     证：证明对于 $c \geq 3/2$, 存在一个 n_0，当$n \geq n_0$ 时$\sum_{k=0}^{n} 3^k \leq c3^n$.

     当 $n = 0$ 时，$\sum_{k=0}^{n} 3^k = 1 \leq c = c3^n$.

     设$n \leq m$ 时成立. 令$n = m + 1$，则

     $\sum_{k=0}^{m+1} 3^k = \sum_{k=0}^{m} 3^k + 3^{m+1} \leq c3^m + 3^{m+1} = c3^{m+1}\left(\frac{1}{3} + \frac{1}{c}\right) \leq c3^{m+1}$.
   - 例2（错误证明）：$\sum_{k=1}^{n} k = O(n)$

     证：$n = 1$ 时，$\sum_{k=1}^{1} k = 1 = \mathrm{O}(1)$，$\sum_{k=1}^{n+1} k = \sum_{k=1}^{n} k + (n+1) = \mathrm{O}(n) + (n+1) = \mathrm{O}(n)$.

     错在$O(n)$ 的常数 c随 n的增长而变化, 不是常数。

     要用数学归纳法证明$\sum_{k=1}^{n} k = O(n)$, 假设存在常数c,对于n成立, 需要证明: 对相同的c, 满足 $\sum_{k=1}^{n+1} k \leq c(n+1)$
2. **直接求解**，通过用级数中最大项的界作为其他项的界
   - $\sum_{k=1}^{n} k \leq \sum_{k=1}^{n} n = n^2$
   - $\sum_{k=1}^{n} a_i \leq n \times \max\{a_k\}$
   - 但当一个级数以**几何级数（指数级数）**增长时，用最大项作为每一项的上界过于宽松，并不理想

     例 3. 设对于所有 $k \geq 0, a_{k+1}/a_k \leq r < 1$, 求 $\sum^{n} a_k$ 的上界.

     $a_1/a_0 \leq r \Rightarrow a_1 \leq a_0 r$ ，

     $a_2/a_1 \leq r \Rightarrow a_2 \leq a_1 r \leq a_0 r^2$,

     $a_3/a_2 \leq r \Rightarrow a_3 \leq a_2 r \leq a_0 r^3$ .....

     $a_k/a_{k-1} \leq r \Rightarrow a_k \leq a_{k-1} r \leq a_0 r^k$

     于是， $\sum_{k=0}^{n} a_k \leq \sum_{k=0}^{\infty} a_0 r^k = a_0 \sum_{k=0}^{\infty} r^k = \frac{a_0}{1-r}$.
   - 例 4. 求$\sum_{k=1}^{\infty}\left(k/3^k\right)$ 的界

     $\frac{k+1}{3^{k+1}}/\frac{k}{3^k} = \frac{1}{3}\cdot\frac{k+1}{k} = \frac{1}{3}\left(1 + \frac{1}{k}\right) \leq \frac{2}{3} = r$. 于是 $\sum_{k=1}^{\infty}\frac{k}{3^k} \leq \sum_{k=0}^{\infty} a_1 r^k = \sum_{k=0}^{\infty}\frac{1}{3}\left(\frac{2}{3}\right)^k = \frac{1}{3}\cdot\frac{1}{1-\frac{2}{3}} = 1$
3. 直接求解，**分割（分裂）求和**
   - 将和式项分割，并忽略其常数个起始项，对常数k_0:
   - 例 5. 求 $\sum_{k=0}^{\infty}\frac{k^2}{2^k}$的上界。

     解: 当$k \geq 3$ 时，$\frac{(k+1)^2/2^{k+1}}{k^2/2^k} = \frac{(k+1)^2}{2k^2} \leq \frac{8}{9}$.于是:

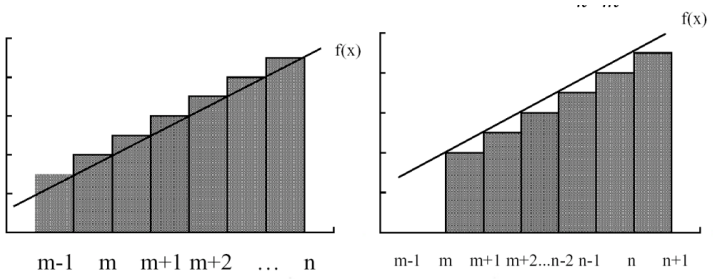
     $\sum_{k=0}^{\infty}\frac{k^2}{2^k} = \sum_{k=0}^{2}\frac{k^2}{2^k} + \sum_{k=3}^{\infty}\frac{k^2}{2^k} \leq O(1) + \sum_{k=3}^{\infty}\frac{9}{8}\left(\frac{8}{9}\right)^k \leq O(1) + \sum_{k=1}^{\infty}\frac{9}{8}\left(\frac{8}{9}\right)^{k-3} = O(1) + \frac{9}{8} * \frac{1}{1-\frac{8}{9}} = O(1)$
   - 例 6. 求$H_n = \sum_{k=1}^{n}\frac{1}{k}$的上界。

     $\sum_{k=1}^{n}\frac{1}{k} = \frac{1}{1} + \left(\frac{1}{2} + \frac{1}{3}\right) + \left(\frac{1}{4} + \frac{1}{5} + \frac{1}{6} + \frac{1}{7}\right) + \cdots\cdots \leq \sum_{i=0}^{\lceil \lg n\rceil}\sum_{j=0}^{2^i-1}\frac{1}{2^i+j} \leq \sum_{i=0}^{\lceil \lg n\rceil}\sum_{j=0}^{2^i-1}\frac{1}{2^i} \leq \sum_{i=0}^{\lceil} 1 \leq \lg n + 1 = C$
4. 积分求和近似
   - 如果f(k)单调递增，则:

$$\int_{m-1}^{n} f(x)dx \leq \sum_{k=m}^{n} f(k) \leq \int_{m}^{n+1} f(x)dx$$

   - 如果f(k)单调递减，则:

$$\int_{m}^{n+1} f(x)dx \leq \sum_{k=m}^{n} f(k) \leq \int_{m-1}^{n} f(x)dx$$



   - 例7: $\sum_{k=1}^{n}\frac{1}{k}$

$\sum_{k=1}^{n}\frac{1}{k}\geq\int_{1}^{n+1'}\frac{dx}{x}=\ln(n+1),\quad\sum_{k=2}^{n}\cdot\frac{1}{k}\leq\int_{1}^{n}\frac{dx}{x}=\ln n$

$\ln(n+1)\leq\sum_{k=1}^{n}\frac{1}{k}\leq\ln n+1$

注意：用不同方法得到的界限可能不一样

递归方程

(39条消息) 算法导论 — 思考题4-3 更多的递归式例子_yangtzhou的博客-CSDN博客

针对如下形式：

$$T(n)=aT\left(n/b\right)+f(n)$$

忽略边界条件：

对于一个常量规模的输入，算法运行时间为常量，对于足够小的n，T(n) 为常量，改变T(1)不会改变函数的增长阶。

替换（代入）法

(39条消息) 算法导论 — 4.3 用代入法求解递归式yangtzhou的博客-CSDN博客代入法求解递归式

首先猜想（界），然后用数学归纳法证明

例：求解 $T(n)=2T(\lfloor n/2\rfloor)+n,T(1)=1$ 的上界

- 根据经验，猜测其解为 $T(n)=O(n\log n)$，要求证明：$\exists C>0,\ T(n)\leq cn\log n$。
- 归纳法证明（第二数学归纳法）：

  1) 假设对于正整数$m<n$都成立，那么$T(\lfloor n/2\rfloor)\leq c\lfloor n/2\rfloor\log(\lfloor n/2\rfloor)$；

  2) 将其代入递归式，得到：

  $T(n)\leq 2(c\lfloor n/2\rfloor\log(\lfloor n/2\rfloor))+n\leq cn\log(n/2)+n$

  $=cn\log n-cn\log 2+n=cn\log n-cn+n\leq cn\log n$，当 $c\geq 1$

- 初始条件不成立时，往后推，看是否成立
- T(1)<clog1=0, 与T(1)=1矛盾
- T(2)= 4<=c 2log2,只需c>2,成立
- 渐进符号只要求：

  ∃ n0 ，当 n ≥ n0时，T(n) ≤ cnlgn。

  对于大多数递归式而言，扩展边界条件使得归纳假设对较小的n成立，是一种简单直接的方法。

- 猜测方法：
  - 联想已知的T(n)

例1. 求解$T(n)$=2T(n/2 + 17) + n

解：猜测：$T(n)=2T\left(\frac{n}{2}+17\right)+n$ 与 $T(n)=2T\left(\frac{n}{2}\right)+n$ 只相差一个 17.

当 n 充分大时 $T\left(\frac{n}{2}+17\right)$ 与 $T\left(\frac{n}{2}\right)$ 的差别并不大，因为

$\frac{n}{2}+17$ 与 $\frac{n}{2}$ 相差小．我们可以猜 $T(n)=O(n\lg n)$.

解：

（1）不验证初始条件了。

（2）假设$m<n$都成立，即

$$T(n)\leq cn\lg n$$

（3）验证$m=n$时，

$$T(n)=2*T\left(\frac{n}{2}+17\right)+n=2c\left(\frac{n}{2}+17\right)*\lg\left(\frac{n}{2}+17\right)+n$$

$=(cn+34c)*(\lg(n+34)-1)+n$当$n$足够大时，有$n+34<1.5n$

$\leq(cn+34c)*(\lg(1.5n)-1)+n$

$=(cn+34c)*(\lg n+\lg 1.5-1)+n$

$=cn\lg n+((\lg 1.5-1)c+1)n+34c(\lg 1.5-1)+34c\lg n$

$\leq cn\lg n$

  - 先证明较松的上下界，然后缩小不确定性范围

例 3．求解 $T(n)=2T\left(\frac{n}{2}\right)+n$.

首先证明T(n)= Ω(n), T(n) = O(n²)，然后逐渐地降低上界，提高下界：

Ω(n)的上一个阶是Ω(nlogn)，O(n²)的下一个阶是O(nlogn).

- **变量替换**：经变量替换把递归方程变换为熟悉的方程

例6. 求解$T(n)=2*T(\sqrt{n})+\lg n$。只考虑$\sqrt{n}$是整数的情形。

解：令$m=\lg n$，则$n=2^m$，$T(2^m)=2*T\left(2^{\frac{m}{2}}\right)+m$

令$S(m)=T(2^m)$，则$T\left(2^{\frac{m}{2}}\right)=S\left(\frac{m}{2}\right)$。

于是，$S(m)=2S\left(\frac{m}{2}\right)+m$。

显然，$S(m)=O(m\lg m)$，即$T(2^m)=O(m\lg m)$。

由于$2^m=n$，$m=\lg n$，$T(n)=O(\lg n\lg(\lg n))$。

- 细微差别的处理：
  - 问题：猜测正确，数学归纳法的归纳步骤似乎证不出来

    解决方法：从guess中减去一个低阶项，使得归纳假设更强，可能work

例 4．求解 $T(n)=T(\lfloor n/2\rfloor)+T\left(\left\lceil\frac{n}{2}\right\rceil\right)+1$

解：(1) 我们猜 $T(n)=\mathrm{O}(n)$

证：$T(n)\le c\lfloor n/2\rfloor+c\lceil n/2\rceil+1=cn+1\ne cn$

证不出 $T(n)=\mathrm{O}(cn)$

(2) 减去一个低阶项，猜 $T(n)\le cn-b$，$b\ge 0$ 是常数

证：设当$\le n-1$时成立

$$T(n)=T(\lfloor n/2\rfloor)+T\left(\left\lceil\frac{n}{2}\right\rceil\right)+1\le c\left\lfloor\frac{n}{2}\right\rfloor-b+c\left\lceil\frac{n}{2}\right\rceil-b+1$$

$$=cn-2b+1=cn-b-b+1\le cn-b\ （只要 b\ge 1）。$$

*c必须充分大，以满足边界条件。

- 为什么假设更强能work：

  证明时可以使用更强更紧的界，比如本例中两次减去常数b,而证明结论只需要减去1次。

迭代（递归树）方法

画出递归树--循环地展开递归方程--把递归方程转化为和式--使用求和技术解之

(35条消息) 算法导论 — 4.4 用递归树方法求解递归式*yangtzhou的博客-CSDN博客*递归树法

【计算机科学】如何计算时间复杂度(Recursion tree & Master method)*哔哩哔哩*bilibili

- 分治思想：通过解一个问题的子问题，来解一个问题

$$T(n)=aT(n/b)+f(n)$$

  - a : 子问题数量
  - b : 子问题从原问题缩小的比例，n/b为子问题规模
  - f(n): 将问题分解和子问题解整合的代价
- 根结点表示递归调用顶层的代价

  内部节点，表示不同层次递归调用产生的代价，即（分解和合并）子问题的代价

  树的分枝数量取决于子问题的数量

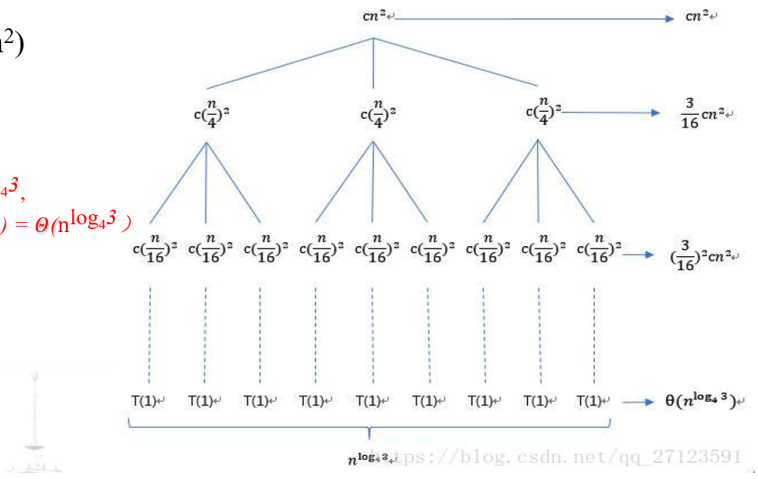
  叶节点表示边界条件值
- 例：

$$T(n)=3T(\lfloor n/4\rfloor)+\Theta(n^2)$$
$$=3T(\lfloor n/4\rfloor)+cn^2$$

树高度：$\log_4 n+1$

最底层叶子节点数：$3^{\log_4 n}=n^{\log_4 3}$，

所以叶子节点总代价：$n^{\log_4 3}T(1)=\Theta(n^{\log_4 3})$



https://blog.csdn.net/qq_27123591

现在将所有层次的代价相加得到整棵树的代价：

$$T(n)=cn^2+\frac{3}{16}cn^2+\left(\frac{3}{16}\right)^2cn^2+\cdots+\left(\frac{3}{16}\right)^{\log_4 n-1}cn^2+\Theta(n^{\log_4 3})$$

$$=\sum_{i=0}^{\log_4 n-1}\left(\frac{3}{16}\right)^i cn^2+\Theta(n^{\log_4 3})=\frac{(3/16)^{\log_4 n}-1}{(3/16)-1}cn^2+\Theta(n^{\log_4 3})$$

最后一个式子看上去有些乱，不过可以将量适当放松，用无限递减等比级数作为上界。回到上一步，应用方程式(A.6)，有

$$T(n)=\sum_{i=0}^{\log_4 n-1}\left(\frac{3}{16}\right)^i cn^2+\Theta(n^{\log_4 3})<\sum_{i=0}^{\infty}\left(\frac{3}{16}\right)^i cn^2+\Theta(n^{\log_4 3})$$

$$=\frac{1}{1-(3/16)}cn^2+\Theta(n^{\log_4 3})=\frac{16}{13}cn^2+\Theta(n^{\log_4 3})=O(n^2)$$

于是，我们得到了原来的递归式 $T(n)=3T(\lfloor n/4\rfloor)+\Theta(n^2)$ 的一个猜测。在这个例子里，cn^2 系数形成一个递减的等比级数，由方程式(A.6)，可知这些系数的总和的上界是常数 16/13。由于树根所需的代价为 cn^2，所以根部的代价占总代价的一个常数部分。换句话说，整棵树的总代价是由根部的代价所决定的。

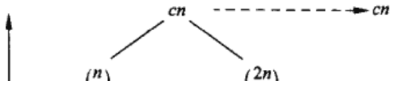
事实上，如果 $O(n^2)$ 确实是此递归式的上界(稍后将会证明)，那么它一定是确界(tight bound)。为什么呢？第一个递归调用需要的代价是 $\Theta(n^2)$，所以 $\Omega(n^2)$ 一定是此递归式的下界。

现在可以使用代换法来验证猜测的正确性，$T(n)=O(n^2)$ 是递归式 $T(n)=3T(\lfloor n/4\rfloor)+\Theta(n^2)$ 的一个上界。我们需要证明，当某常数 $d>0$，$T(n)\leqslant dn^2$ 成立。适用与前面相同的常数 $c>0$，有

$$T(n)\leqslant 3T(\lfloor n/4\rfloor)+cn^2\leqslant 3d\lfloor n/4\rfloor^2+cn^2\leqslant 3d(n/4)^2+cn^2$$

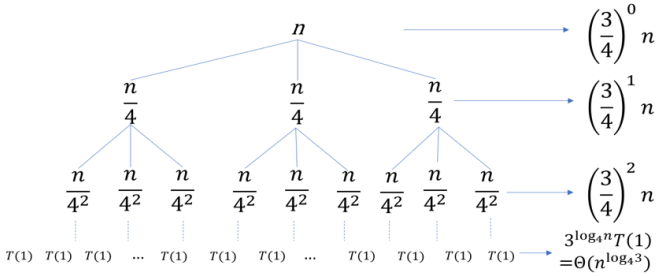
$$=\frac{3}{16}dn^2+cn^2\leqslant dn^2$$

只要 $d\geqslant(16/13)c$，最后一步都会成立。



- 迭代递归树方法提供了一种简单直接的猜想界的方法!
- 例:

$$T(n)=n+3T\left(\lfloor n/4\rfloor\right)$$



$$=n+3\left(\lfloor n/4\rfloor+3T\left(\lfloor n/16\rfloor\right)\right)$$

$$=n+3\left(\lfloor n/4\rfloor+3\left(\left\lfloor\frac{n}{16}\right\rfloor+3T\left(\lfloor n/64\rfloor\right)\right)\right)$$

$$=n+3\lfloor n/4\rfloor+9\left\lfloor\frac{n}{16}\right\rfloor+27T\left(\lfloor n/64\rfloor\right)$$

$$=n+3\lfloor n/4\rfloor+3^2\left\lfloor\frac{n}{4^2}\right\rfloor+3^3\left(\left\lfloor\frac{n}{4^3}\right\rfloor\right)+\cdots+3^i T\left(\lfloor n/4^i\rfloor\right)$$

令 $\frac{n}{4^i}=1\Rightarrow 4^i=n\Rightarrow i=\log_4 n$

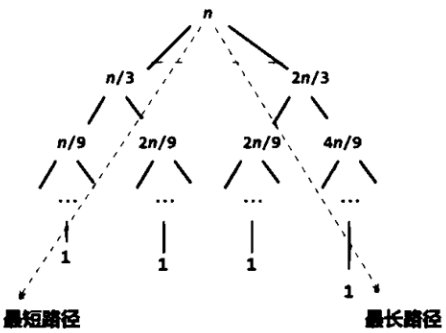
$$=n+3\lfloor n/4\rfloor+3^2\left\lfloor\frac{n}{4^2}\right\rfloor+3^3\left(\left\lfloor\frac{n}{4^3}\right\rfloor\right)+\cdots+3^{\log_4 n}T(\lfloor 1\rfloor)$$

$$\leq\sum_{i=0}^{\log_4 n}\left(\frac{3}{4}\right)^i n+O(n)\leq n\sum_{i=0}^{\infty}\left(\frac{3}{4}\right)^i+O(n)=n\times\frac{1}{1-3/4}+O(n)$$

$$=4n+O(n)=O(n)$$

- 例:

$$T(n)=1$$
$$T(n)=T(n/3)+T(2n/3)+n$$



- 设最长路径的长度为h，有$n(2/3)^h=1$。
- 求出$h=\log_{3/2}n$，因此这棵递归树有$\log_{3/2}n$层，每层结点的数值和为n。
- $T(n)=O(n\log_{3/2}n)=O(n\log_2 n)$。

直觉上，我们预期递归式的解至多是层数乘以每层的代价，也就是 $O(cn\log_{3/2}n)=O(n\lg n)$。总代价被均匀地分布到递归树内的每一层上。这里还有一个复杂点：我们还没有考虑叶子的代价。如果这棵树是高度为 $\log_{3/2}n$ 的完整二叉树，那么有 $2^{\log_{3/2}n}=n^{\log_{3/2}2}$ 个叶子。由于叶子代价是常数，因此所有叶子代价的总和为 $\Theta(n^{\log_{3/2}2})$，或者说 $\omega(n\lg n)$。然而，这棵递归树并不是完整的二叉树，少于 $n^{\log_{3/2}2}$ 个叶子，而且从树根往下的过程中，越来越多的内部结点在消失。因此，并不是所有层次都刚好需要 cn 代价；越靠近底层，需要的代价越少。我们可以计算出准确的总代价，但记住我们只是想要找出一个猜测来使用到代换法中。让我们容忍这些误差，而来证明上界为 $O(n\lg n)$ 的猜测是正确的。

事实上，可以用代换法来证明 $O(n\lg n)$ 是递归式解的上界。下面证明 $T(n)\leqslant dn\lg n$，当 d 是一个合适的正值常数，有

$$\begin{aligned}T(n)&\leqslant T(n/3)+T(2n/3)+cn\\&\leqslant d(n/3)\lg(n/3)+d(2n/3)\lg(2n/3)+cn\\&=(d(n/3)\lg n-d(n/3)\lg 3)+(d(2n/3)\lg n-d(2n/3)\lg(3/2))+cn\\&=dn\lg n-d((n/3)\lg 3+(2n/3)\lg(3/2))+cn\\&=dn\lg n-d((n/3)\lg 3+(2n/3)\lg 3-(2n/3)\lg 2)+cn\\&=dn\lg n-dn(\lg 3-2/3)+cn\leqslant dn\lg n\end{aligned}$$

上式成立的条件是 $d\geqslant c/(\lg 3-(2/3))$。因此，没有必要去更准确地计算递归树中的代价。

Master定理（主定理、主函数）方法

(39条消息) 算法导论 — 4.5 用主方法求解递归式_yangtzhou的博客-CSDN博客

求解递归式$T(n)=aT(n/b)+f(n)$的复杂度，$a\geq 1,b>1$是常数。

Master定理:

设 $a\geq 1$ 和 $b>1$是常数,$f(n)$是一个函数，$T(n)$是定义在非负整数集上的函数,$T(n)=aT(n/b)+f(n)$。

那么 $T(n)$ 可以如下求解：

- 若 $f(n)=O\left(n^{\log_b a-\varepsilon}\right),\varepsilon>0$ 是常数, 则 $T(n)=\Theta\left(n^{\log_b a}\right)$
- 若 $f(n)=\Theta\left(n^{\log_b a}\right)$（同阶），则 $T(n)=\Theta\left(n^{\log_b^a}\lg n\right)=\Theta(f(n)\lg n)$。
- 若 $f(n)=\Omega\left(n^{\log_b a+\varepsilon}\right),\varepsilon>0$是常数, 且对某个常数 $c<1$和所有充分大的 n **（正则条件）**，有 $af(n/b)\leq cf(n)$, 则 $T(n)=\Theta(f(n))$

更进一步：

- 在第一种情况,$f(n)$ 不仅小于$n^{\log_b^a}$，必须多项式地小于，即对于一个常数$\varepsilon>0, f(n)=O\left(\frac{n^{\log_b^a}}{n^\varepsilon}\right)$。
- 在第三种情况, $f(n)$不仅大于$n^{\log_b^a}$，必须多项式地大于，即对于一个常数$\varepsilon>0,\quad f(n)=\Omega\left(n^{\log_b^a}\cdot n^\varepsilon\right)$。

直观地：我们用 $f(n)$ 与 $n^{\log_b a}$ 比较:

- 若$n^{\log_b^a}$大，则$T(n)=\Theta(n^{\log_b^a})$
- 若 $f(n)$ 大, 则$T(n)=\Theta(f(n))$
- 若 $f(n)$与 $n^{\log_b^a}$ 同阶, 则 $T(n)=\Theta\left(n^{\log_b^a}\lg n\right)=\Theta(f(n)\lg n)$

特殊形式:

$$\text{对于以下递推式：}T(n)=aT(n/b)+O\left(n^d\right)\quad(a>=1\text{ 且 }b>1)$$

$$\text{渐进界为：}T(n)=\begin{cases}O\left(n^d\log(n)\right) & \text{if } a=b^d \quad(\log_b a=d)\\ O\left(n^d\right) & \text{if } a<b^d \quad(\log_b a<d)\\ O\left(n^{\log_b(a)}\right) & \text{if } a>b^d \quad(\log_b a>d)\end{cases}$$

例：求解$T(n)=3T(n/4)+n\lg n$

解：$a=3,b=4,f(n)=n\lg n,n^{\log_b^a}=n^{\log_4^3}=O\left(n^{0.793}\right)$

$(1)f(n)=n\lg n\geq n^{\log_b^a+\varepsilon},\varepsilon\approx 0.2$

(2)对所有 $n,af(n/b)=3\times\frac{n}{4}\lg\frac{n}{4}=\frac{3}{4}n\lg\frac{n}{4}\leq\frac{3}{4}n\lg n=cf(n),\quad c=\frac{3}{4}$

于是 $T(n)=\Theta(f(n))=\Theta(n\lg n)$。

例：求解 $T(n)=2T(n/2)+n\lg n$

解：$a=2,b=2,f(n)=n\lg n,n^{\log_b^a}=n^{\log_2^2}=nf(n)=n\lg n$大于 $n^{\log_b^a}=n$, 但**不是多项式地大于（指两者比值的商是一个幂函数n^ε，而不是一个**$\lg n$），Master 定理不适用于该$T(n)$。

习题

1. 已知两个长度分别为m和n的升序链表，若将他们合并为一个长度为$m+n$的降序链表，则最坏情况下的时间复杂度是：
   $\Omega(max(m,n))$
2. $f(n)+o(f(n))=\Theta(f(n))$ 正确